

OBJECT ORIENTED PROGRAMMING WITH C++

DSA0103

CO3-AT1 – Analytical Problem Solving

NAME : Ranjitha. R

REG NO : 192421295

Question 1: Banking Account Management System

A banking application needs to create and manage customer account objects. The system should support different ways of initializing an account depending on the information available.

Design a C++ program using a class Account with the following constructors:

Default constructor – initializes an account with default values.

Parameterized constructor – initializes account number, customer name, and balance.

Copy constructor – creates a new account by copying the details of an existing account.

Tasks:

1. Define the Account class with suitable data members.
2. Implement the default, parameterized, and copy constructors.
3. Create objects using each constructor.
4. Display the details of each account.
5. Analyze and explain how object initialization differs for the three constructors.
6. Create a new account object from an existing account object using the copy constructor.
7. Predict and explain the order in which constructors are called during object creation and copying.

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
class Account {
```

```
    int accountNo;
```

```
    string name;
```

```
    float balance;
```

```
public:
```

```
    Account() {
```

```
        accountNo = 0;
```

```
        name = "Unknown";
```

```
        balance = 0;
```

```

    }

    Account(int no, string n, float b) {
        accountNo = no;
        name = n;
        balance = b;
    }

    Account(const Account &a) {
        accountNo = a.accountNo;
        name = a.name;
        balance = a.balance;
    }

    void display() {
        cout << "Account No: " << accountNo << endl;
        cout << "Name: " << name << endl;
        cout << "Balance: " << balance << endl;
        cout << endl;
    }
};

int main() {
    Account a1;
    Account a2(101, "Ranjitha", 5000);
    Account a3 = a2;
    cout << "Default Account:\n";
    a1.display();
    cout << "Parameterized Account:\n";
    a2.display();
    cout << "Copied Account:\n";
    a3.display();
    return 0;
}

```

OUTPUT:

```
1 #include <iostream>
2 using namespace std;
3
4 class Account {
5     int accountNo;
6     string name;
7     float balance;
8
9 public:
10    // Default constructor
11    Account() {
12        accountNo = 0;
13        name = "Unknown";
14        balance = 0;
15    }
16
17    // Parameterized constructor
18    Account(int no, string n, float b) {
19        accountNo = no;
20        name = n;
21        balance = b;
22    }
23
24    // Copy constructor
25    Account(const Account &a) {
26        accountNo = a.accountNo;
27        name = a.name;
28        balance = a.balance;
29    }
30
31    void display() {
32        cout << "Account No: " << accountNo << endl;
33        cout << "Name: " << name << endl;
34        cout << "Balance: " << balance << endl;
35        cout << endl;
36    }
37 };
38
39 int main() {
40     Account a1; // Default
41     Account a2(101, "Ranjitha", 5000); // Parameterized
42     Account a3 = a2; // Copy
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	4.016 Mb

Your Output

Default Account:
Account No: 0
Name: Unknown
Balance: 0

Parameterized Account:
Account No: 101
Name: Ranjitha
Balance: 5000

Question 2: Inventory Product Management System

A software company is developing an inventory management system using a class called Product. The system should allow products to be created, copied, combined, and removed from memory. Write a complete C++ program for the Inventory Product Management System using a parameterized constructor, copy constructor, destructor, and overloaded + operator. Demonstrate the complete object lifecycle through suitable output statements

Analyze:

1. Design the Product class with appropriate data members.
2. Implement a parameterized constructor to initialize product details.
3. Implement a copy constructor to duplicate a product.
4. Implement a destructor to release resources when an object is destroyed.
5. Overload the + operator to add the stock quantities of two products.
6. Create two product objects using the parameterized constructor.
7. Create another product using the copy constructor.
8. Add two products using the overloaded + operator.
9. Display the product details and resulting stock quantity.
10. Trace and explain the sequence of constructor and destructor calls.

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
class Product {
```

```
    string name;
```

```
    int quantity;
```

```
    float price;
```

```
public:
```

```
    Product(string n, int q, float p) {
```

```
        name = n;
```

```

        quantity = q;
        price = p;
        cout << "Constructor called for " << name << endl;
    }
    Product(const Product &p) {
        name = p.name;
        quantity = p.quantity;
        price = p.price;
        cout << "Copy constructor called for " << name << endl;
    }
    ~Product() {
        cout << "Destructor called for " << name << endl;
    }
    Product operator+(const Product &p) {
        return Product(name + " + " + p.name,
            quantity + p.quantity,
            price + p.price);
    }
    void display() {
        cout << "Product: " << name << endl;
        cout << "Quantity: " << quantity << endl;
        cout << "Price: " << price << endl;
        cout << endl;
    }
};

int main() {
    Product p1("Laptop", 5, 50000);
    Product p2("Mouse", 10, 500);
    Product p3 = p1;    // Copy constructor
    Product p4 = p1 + p2; // + operator

```

```

    cout << "\nProduct 1:\n";

    p1.display();

    cout << "Product 2:\n";

    p2.display();

    cout << "Copied Product:\n";

    p3.display();

    cout << "Combined Product:\n";

    p4.display();

    return 0;

}

```

OUTPUT:

```

1 #include <iostream>
2 using namespace std;
3
4 class Product {
5     string name;
6     int quantity;
7     float price;
8
9 public:
10    // Parameterized constructor
11    Product(string n, int q, float p) {
12        name = n;
13        quantity = q;
14        price = p;
15        cout << "Constructor called for " << name << endl;
16    }
17
18    // Copy constructor
19    Product(const Product &p) {
20        name = p.name;
21        quantity = p.quantity;
22        price = p.price;
23        cout << "Copy constructor called for " << name << endl;
24    }
25
26    // Destructor
27    ~Product() {
28        cout << "Destructor called for " << name << endl;
29    }
30
31    // + operator overloading
32    Product operator+(const Product &p) {
33        return Product(name + " + " + p.name,
34                        quantity + p.quantity,
35                        price + p.price);
36    }
37
38    void display() {
39        cout << "Product: " << name << endl;
40        cout << "Quantity: " << quantity << endl;
41        cout << "Price: " << price << endl;
42        cout << endl;
43    }
44 };

```

Enter input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	3.824 Mb

Your Output

```

Constructor called for Laptop
Constructor called for Mouse
Copy constructor called for Laptop
Constructor called for Laptop + Mouse

Product 1:
Product: Laptop
Quantity: 5
Price: 50000

```

Question 3: Shopping Cart Management System

An online shopping application maintains customer shopping carts. A cart contains multiple products and their quantities. The system should support creating a cart, duplicating an existing cart, and merging two carts. Write a complete C++ program to implement the ShoppingCart system. Demonstrate parameterized construction, copy construction, cart merging using +, and destruction of objects.

Tasks:

- Design the Product class with appropriate data members.
- Implement a parameterized constructor to initialize product details.
- Implement a copy constructor to duplicate a product.
- Implement a destructor to release resources when an object is destroyed.
- Overload the + operator to add the stock quantities of two products.
- Create two product objects using the parameterized constructor.
- Create another product using the copy constructor.
- Add two products using the overloaded + operator.
- Display the product details and resulting stock quantity.
- Trace and explain the sequence of constructor and destructor calls.
- Explain what may happen if dynamically allocated memory is not released in the destructor

CODE:

```
#include <iostream>

using namespace std;

class Product {
    string name;
    int quantity;
    float price;
public:
    Product(string n, int q, float p) {
        name = n;
        quantity = q;
        price = p;
        cout << "Constructor: " << name << endl;
    }
    Product(const Product &p) {
        name = p.name;
        quantity = p.quantity;
        price = p.price;
        cout << "Copy Constructor: " << name << endl;
    }
    Product operator+(const Product &p) {
        return Product(name + " + " + p.name,
            quantity + p.quantity,
            price + p.price);
    }
    ~Product() {
        cout << "Destructor: " << name << endl;
    }
    void display() {
```

```

        cout << "Product: " << name << endl;

        cout << "Quantity: " << quantity << endl;

        cout << "Price: " << price << endl;

    }

};

int main() {

    Product p1("Laptop", 2, 50000);

    Product p2("Laptop", 3, 50000);

    Product p3 = p1;    // Copy constructor

    Product p4 = p1 + p2; // Merge products

    cout << "\nProduct 1:\n";

    p1.display();

    cout << "\nProduct 2:\n";

    p2.display();

    cout << "\nCopied Product:\n";

    p3.display();

    cout << "\nMerged Product:\n";

    p4.display();

    return 0;

}

```

OUTPUT:

The image shows a C++ program and its execution output. The source code on the left defines a `Product` class with a parameterized constructor, a copy constructor, an addition operator, and a `display` method. The `main` function creates two `Product` objects, copies one, and adds them together, then displays each object's details.

The output on the right shows the program executed successfully. It displays the constructor messages for each object, the copied object, and the merged object. Finally, it shows the details for Product 1: Product: Laptop, Quantity: 2, Price: 50000.

```

1 #include <iostream>
2 using namespace std;
3
4 class Product {
5     string name;
6     int quantity;
7     float price;
8
9 public:
10    // Parameterized constructor
11    Product(string n, int q, float p) {
12        name = n;
13        quantity = q;
14        price = p;
15        cout << "Constructor: " << name << endl;
16    }
17
18    // Copy constructor
19    Product(const Product &p) {
20        name = p.name;
21        quantity = p.quantity;
22        price = p.price;
23        cout << "Copy Constructor: " << name << endl;
24    }
25
26    // + operator overloading
27    Product operator+(const Product &p) {
28        return Product(name + " + " + p.name,
29            quantity + p.quantity,
30            price + p.price);
31    }
32
33    // Destructor
34    ~Product() {
35        cout << "Destructor: " << name << endl;
36    }
37
38    void display() {
39        cout << "Product: " << name << endl;
40        cout << "Quantity: " << quantity << endl;
41        cout << "Price: " << price << endl;
42    }
43 };

```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	3.952 Mb

Your Output

```

Constructor: Laptop
Constructor: Laptop
Copy Constructor: Laptop
Constructor: Laptop + Laptop

Product 1:
Product: Laptop
Quantity: 2
Price: 50000

```